

AEGIS



Adaptive Exam Guardian and Integrity System

A browser-native anti-cheat exam portal that monitors student behaviour in real time using privacy-minimised telemetry, scores six signals into a confidence score, and surfaces integrity reports for human review — without webcams, browser extensions, or invasive surveillance.

BACKEND

FastAPI · Auth

REST · WebSocket

SIGNAL SCORER

Rule Engine

Scoring · Alerts

ML & EVALUATION

Baseline · Metrics

XGBoost · SHAP

DATABASE & API

SQLAlchemy · ORM

Alembic · Tests

DEVOPS & GDPR

CI/CD · Azure

Bicep · Actions

FRONTEND & SDK

THE PROBLEM

Online Exams Face a New Threat

Generative AI changes everything

A student can paste a question into ChatGPT and receive a fluent, structured answer in seconds — undetectable from genuine effort by text alone.

Webcam proctoring

Invasive surveillance · setup friction · high false-positive rate · not designed for AI-assisted cheating

AI text detectors

OUR POSITIONING

How We Fit

Approach	Privacy	AI-aware	Evidence
Webcam proctoring	✗	✗	⚠
AI text detectors	✓	⚠	✗
Locked browsers	⚠	✗	✗
AEGIS	✓	✓	✓

ONE-SENTENCE POSITION

"A browser-native exam portal that combines behavioural telemetry, adaptive question rendering, and human-reviewed confidence scores to give professors evidence of suspicious exam behaviour — without invasive surveillance."

Three Pillars of AEGIS



Privacy-first Telemetry

Browser-native SDK captures tab visibility, paste events, keystroke intervals, viewport changes, and answer timing.

- ✓ Event metadata only — no key content, no clipboard text, no recordings



Explainable Scoring

Six weighted signals combine into a 0–1 confidence score. Rule-based heuristics in MVP; XGBoost after pilot data.

- 💡 Every threshold is documented — no black-box scores



Human Review

Professor console shows a live event timeline, confidence score, and flagged events. No automatic verdicts.

- ⚠️ The system produces evidence — professors make the decision

6

Behavioural signals

0–1

Confidence score

0

Automatic verdicts

90d

Data retention limit

Who Benefits and How



Professor — Dr Sarah

1. Creates quiz with MCQ + short-answer questions
2. Toggles Question 3 to **image-only format** (no DOM text)
3. Schedules exam session, shares student link
4. Sees **live cohort view** during exam — per-student event feed
5. Reviews integrity report post-exam: confidence score + flagged event timeline
6. Makes an informed, defensible review decision



Pain point solved: *"I need evidence to support a review decision, not just a suspicious answer."*



Student — James

1. Logs in with UCD email
2. Reads **GDPR transparency screen** — exactly what is collected and why
3. Ticks acknowledgement and enters isolated exam shell
4. Answers questions normally — monitoring runs silently
5. Submits via REST — **fully independent of WebSocket monitoring**
6. Can request own event tape at any time

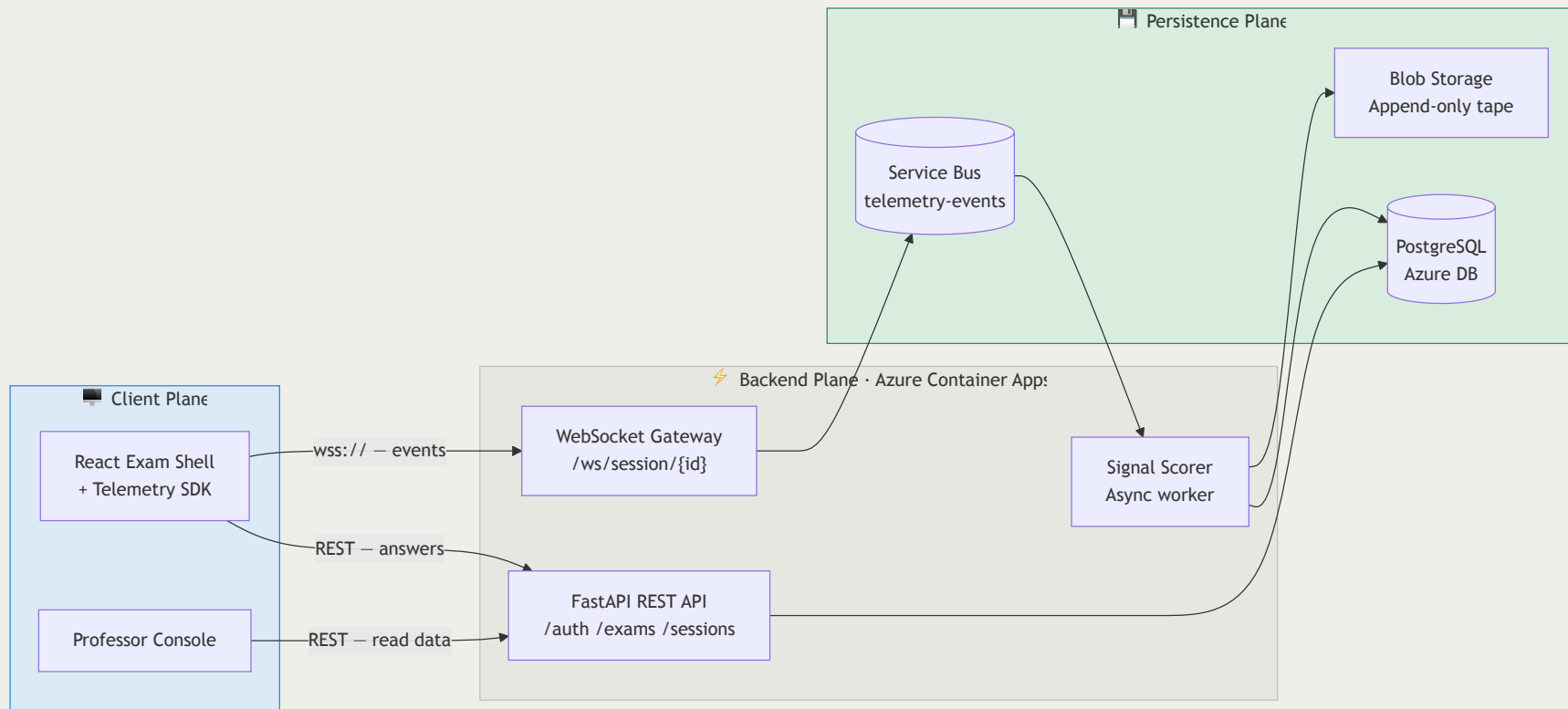


Pain point addressed: *"I shouldn't be penalised for switching tabs to check the time."*

CHAPTER 2

Architecture & Technology

Three-Plane Architecture



Monitoring Never Blocks Submission

✓ REST path (answers)

```
POST /student-sessions/{id}/submit
```

Validates JWT → stores answers → closes session. Zero dependency on WebSocket or scorer.

📡 WebSocket path (telemetry)

```
wss:// /ws/session/{student_session_id}
```

Events buffered locally (circular buffer, 500 events). Reconnects with exponential backoff. Flushed on reconnect.

💡 WebSocket failure test: kill the WS mid-exam, submit via REST →

Infrastructure

Azure Container Apps

FastAPI + React containers · auto-scale · WebSocket-capable · KEDA scaler

Azure DB for PostgreSQL

Flexible Server B1ms · ACID · JSONB event payloads · Alembic migrations

Azure Service Bus Standard

telemetry-events queue · partitioned by student_id · KEDA autoscale scorer

Azure Blob Storage

Append-only event tape · tamper-evident · 90-day retention · LRS

Azure Container Registry

Docker images · tagged by git SHA · CI/CD push on merge to main

Azure Bicep IaC

Justified Choices — Alternatives Rejected

Layer	Chosen	Why
Backend	FastAPI (Python 3.12)	Native async/await + WebSocket; Pydantic v2 event validation; auto OpenAPI docs
Frontend	React 18 + TypeScript	Isolated exam shell route; type safety for telemetry SDK; team familiarity
Database	PostgreSQL 16	ACID guarantees on answers; JSONB for event payloads; window functions for baselines
Event queue	Azure Service Bus	Managed PaaS; 50k events/exam at pennies; KEDA integration
Containers	Azure Container Apps	Auto-scale; WebSocket support; no K8s overhead
IaC	Azure Bicep	Simpler than Terraform; native Azure integration

✗ Alternatives rejected

Django — sync-first ORM, not designed for WebSocket-heavy workloads

.NET 8 — technically strong but 4/6 members unfamiliar (13-day risk)

Kafka — cluster overhead unjustified at 75k events/exam

AKS — too much Kubernetes config in 13 days

SQLite — weak concurrency for multi-user exam sessions

Local dev (one command)

```
docker compose up --build
```

```
# PostgreSQL · Service Bus Emulator
```

```
# FastAPI (port 8000) · React (5173)
```


CHAPTER 3

Privacy & Signals

What We Collect — and What We Don't

✓ Collected — event metadata only

⚠ Explicitly removed from SDK

Signal	Collected	Not collected
Tab blur/focus	✓ Timestamps	✗ Destination app
Paste event	✓ Timestamp + char delta	✗ Clipboard content
Keystroke timing	✓ Interval in ms	✗ Which keys pressed
Viewport resize	✓ Width × height	✗ Other windows
Answer timing	✓ Time-to-submit per Q	✗ Off-platform activity

- ✗ `navigator.clipboard.readText()` — removed; requires browser permission dialog; unreliable cross-browser
- ✗ **Screen recording** — not attempted
- ✗ **Webcam / microphone** — not attempted
- ✗ **DevTools detection** — unreliable, removed from scope

GDPR Legal Basis

Dual basis: **Legitimate Academic Interest** (Art 6(1)(f)) + **Explicit Informed Consent** (Art 6(1)(a))

Retention: **90 days** after exam close, then automated purge

Rights: students can **request their event tape** through the portal

Ethics approval: obtained from UCD for pilot study

Six Signals → One Confidence Score

+0.30

Tab-blur before first keypress

Student leaves tab before meaningful input →
document.visibilitychange

+0.25

Paste burst of formatted text

> 200 char delta + markdown patterns (` `` , **, #) → paste event
metadata

+0.20

IKI outlier vs. baseline

z-score of inter-keystroke interval > 3σ from per-student
baseline (first 5 min)

+0.10

Time-to-first-keypress anomaly

< 3 s on a complex question → answer was prepared externally

+0.10

Answer time uniformly low

Suspiciously uniform time-to-submit across questions → reading
from source

Score tiers

0.00 - 0.29	✓	Low risk
0.30 - 0.64	⚠	Review recommended
0.65 - 1.00	🚨	Professor alert

Phase 1 (MVP) — Rule-based

Every threshold is documented and justifiable without training data.
Fully transparent to professors.

Phase 2 (Post-interim) — XGBoost

Once labelled pilot sessions exist: XGBoost classifier on extracted
features. Rules become interpretable baseline features. SHAP
values for explainability.

💡 No single signal alone triggers a flag. Professors see the raw event
timeline alongside the score.

CHAPTER 4

Evaluation

Four-Dimension Evaluation

1 — Detection Accuracy

Controlled pilot study: **honest cohort** vs. **AI-assisted cohort** (using ChatGPT on specified questions).

Metrics: **Precision · Recall · F1 · AUC-ROC**

Target: Precision ≥ 0.80 · Recall ≥ 0.75 · AUC ≥ 0.80

3 — Professor Usability (SUS)

Structured task study with ≥ 5 professor participants. Tasks: create exam, review two contrasting reports.

Instrument: **System Usability Scale (SUS)**

Target: SUS ≥ 70 ("Acceptable") · Aim ≥ 80 ("Good")

2 — False Positive Rate

Measured on the honest cohort. A high FPR erodes professor trust and unfairly flags students.

Target: **FPR $\leq 3\%$** (≤ 3 false flags per 100 honest sessions)

4 — System Performance

Load test with Locust: 100 concurrent WebSocket connections, measure submission latency.

Submission p99 latency: **< 800 ms** @ 100 concurrent users

WS event-to-DB delay: **< 500 ms**

WS failure isolation: **submission still succeeds**

💡 We use AUC-ROC (not accuracy) because the optimal threshold is not predetermined — AUC measures performance across all thresholds and makes

CHAPTER 5

Team, Status & Delivery

AI in Development and in the System

AI as Development Tooling

GitHub Copilot — code completion; boilerplate (routers, Pydantic schemas, test fixtures, Bicep templates). All suggestions read and reviewed before commit.

Claude & ChatGPT — architecture brainstorming, debugging, documentation drafting. Significant use logged in AI Usage Log.

Rule: No AI-generated code merges without the responsible member being able to explain every line. Verified in "defend your code" segments.

AI Components in AEGIS

Phase 1 (MVP) — Rule-based signal scorer. No training data needed. Every threshold documented and explainable.

Phase 2 (post-interim) — XGBoost classifier on labelled pilot sessions. SHAP values for per-prediction explainability. Trained only after sufficient labelled data.

Stylometry (roadmap) — sentence-transformers + ONNX for embedding comparison against prior coursework.

Key Risks & Mitigations

False positives → per-student baseline calibration + FPR target $\leq 3\%$

Training bias → diverse pilot cohort, document demographics

Interpretability → SHAP values + raw event timeline always shown

Gaming → multi-signal requirement + image-only question format

CURRENT STATUS

MVP Sprint Progress



Done

- GitHub monorepo + branch protection (AEGIS-12)
- Docker Compose full local stack (AEGIS-22)
- GitHub Actions CI (lint + test on PR) (AEGIS-24)
- Secrets management — Key Vault refs (AEGIS-25)
- JWT auth — register + login (AEGIS-26)
- SQLAlchemy 2 ORM models × 10 (AEGIS-30)
- Alembic migration + autogenerate (AEGIS-31)
- Quiz creation API — MCQ + short answer (AEGIS-33)



In Progress / Review

- RBAC middleware (AEGIS-27)
- Course enrollments API (AEGIS-28)
- Professor quiz creation UI (AEGIS-36)
- Student exam shell — answering (AEGIS-39)
- Azure resource group + cost alerts (AEGIS-21)
- Pre-commit hooks (ruff, eslint) (AEGIS-23)
- Dev seed script (AEGIS-32)



Upcoming (Sprint 3–4)

- Countdown timer + auto-submit (AEGIS-40)
- Telemetry SDK scaffold (AEGIS-42)
- tab_blur signal (AEGIS-43)
- Paste signal (AEGIS-44)
- IKI keystroke signal (AEGIS-45)
- WebSocket gateway (AEGIS-48)
- Service Bus consumer (AEGIS-51)
- Signal scorer engine (AEGIS-52)
- Professor console — live feed (AEGIS-50)
- Azure Container Apps deploy (AEGIS-16)

Roles, Workstreams & Sprint Plan

Member	Role	Sprint Focus
M1 (Lead)	Backend · Azure · WS gateway	FastAPI, JWT, WebSocket, Docker, CI/CD, Azure
M2	Signal Scorer	Rule engine, Service Bus consumer, scoring tests
M3	ML · Evaluation	Baseline calibration, evaluation metrics, pilot analysis
M4	DB · API	SQLAlchemy models, Alembic, quiz/session API, integration tests
M5	DevOps · GDPR	Docker, GitHub Actions, Azure Bicep, GDPR screen, testing
M6	React · Telemetry SDK	Exam shell, professor console, SDK, demo recording

Sprint Timeline

Jun 6–8**Sprint 1 — Foundation**

Monorepo, Docker Compose, auth end-to-end, Alembic,
React skeleton, CI/CD

Jun 9–12**Sprint 2 — Core Exam Loop**

Quiz API, student exam shell, GDPR screen, WebSocket
gateway, Telemetry SDK

Jun 13–15**Sprint 3 — Professor Console + Azure**

Signal scorer, professor live console, Azure Container
Apps deploy

Jun 16–18**Sprint 4 — Polish + Demo**

Seeded demo data, integration testing, video recording ×
2, Brightspace upload

 **Show-and-Tell #2:** Jun 8 — architecture + skeleton demo

 **Show-and-Tell #3:** Jun 15 — full E2E on Azure

 **Hard deadline:** Jun 19, 23:59 Dublin — upload to Brightspace

How We Responded to the Five Concerns

Concern 1 — ML is overkill

✅ **Agreed.** MVP uses rule-based heuristics with documented thresholds. XGBoost added post-interim when labelled data exists. Rules are actually more defensible at the interim — every threshold has a justification.

Concern 2 — GDPR keylogging risk

✅ **Designed in from Day 1.** We record inter-keystroke intervals (timing) not key content. Clipboard read removed from SDK entirely. Explicit consent screen before every exam. 90-day retention, student data access rights, UCD ethics approval.

Concern 3 — Browser API feasibility

✅ **Verified in feasibility spike.** visibilitychange ✓, paste event ✓, keydown timestamps ✓, resize ✓ — all cross-browser (Chrome, Firefox, Safari, Edge). clipboard.readText() removed; DevTools detection removed as unreliable.

Concern 4 — Azure too early

✅ **Staged introduction.** Weeks 1–2: Docker Compose only, no Azure. Week 2: Azure introduced after core functionality proven locally. Service Bus added alongside the scorer (not before).

Concern 5 — Stress testing expectations unclear

✅ **Clarified with module team.** No formal load testing required for interim. We document scalability design: stateless FastAPI containers behind Container Apps, Service Bus decouples ingestion from scoring, designed for p99 < 800ms @ 500 concurrent students. Full Locust load test (100 concurrent WS connections) planned for the final.

AEGIS

"A privacy-first academic integrity system that gives professors evidence for human review — not automatic verdicts. Evidence, not accusation."

WHAT WE'VE BUILT

Auth • DB • Quiz API • Docker • CI/CD • GDPR

BUILDING NOW

Exam Shell • WebSocket • Telemetry SDK

SPRINT 3–4

Scorer • Console • Azure Deploy • Demo



Hard deadline: 19 June 2026, 23:59 Dublin — upload to Brightspace

Ready for questions